



Attention Please: What Transformer Models Really Learn for Process Prediction

Martin Käppel^(✉) , Lars Ackermann , Stefan Jablonski, and Simon Härtl

Institute for Computer Science, University of Bayreuth, Bayreuth, Germany
{martin.kaeppel,lars.ackermann,stefan.jablonski,
simon.haertl}@uni-bayreuth.de

Abstract. Predictive process monitoring aims to support the execution of a process during runtime with various predictions about the further evolution of a process instance. In the last years a plethora of deep learning architectures have been established as state-of-the-art for different prediction targets, among others the transformer architecture. The transformer architecture is equipped with a powerful attention mechanism, assigning attention scores to each input part that allows to prioritize most relevant information leading to more accurate and contextual output. However, deep learning models largely represent a black box, i.e., their reasoning or decision-making process cannot be understood in detail. This paper examines whether the attention scores of a transformer based next-activity prediction model can serve as an explanation for its decision-making. We find that attention scores in next-activity prediction models can serve as explainers and exploit this fact in two proposed graph-based explanation approaches. The gained insights could inspire future work on the improvement of predictive business process models as well as enabling a neural network based mining of process models from event logs.

Keywords: Predictive Process Monitoring · Transformer · Attention Mechanism · Explainability

1 Introduction

Predictive Business Process Monitoring (PBPM) provides runtime support for the execution of a process with various predictions about the further evolution of a process instance, based on predictive models created from event logs [4]. Early knowledge of the likely course of a process instance offers significant advantages: upcoming steps can be prepared, and potential problems can be identified and mitigated early, e.g., in resource and time planning [4]. In the last years a plethora of deep learning architectures have been established as state-of-the-art for different prediction targets (e.g., next activity, outcome, remaining time), among others Convolutional Neural Networks [14], Long Short Term Memory Neural Networks (LSTM) [2, 6], or more recently transformer architectures [1].

However, deep learning models largely represent a black box, i.e., their reasoning or decision-making process is hidden [13]. Consequently, users often do not trust them, which hinders their application [3]. To address this issue, the discipline of *explainable artificial intelligence (XAI)* tries to find explanations for the decisions of machine learning models [3, 13]. Such approaches also emerged in the context of PBPM [22]. Depending on whether an explainer aims to explain the prediction for a single running process instance or a collection of instances (e.g., an event log) we distinguish between *local* and *global explainers*. While global approaches investigate, how a model makes decisions in general, local approaches examine only individual predictions for a particular input. Previous research has mainly focused on local explainers, so that it still remains unclear, whether deep learning models implicitly learn the underlying process from the event log. The landmark work of Evermann et al. [6] proposes the hypothesis that deep learning models for next activity prediction implicitly learn the process structure. Peeperkorn et al. narrowed this research gap by investigating this hypothesis for LSTMs and revealed that they “*can struggle to learn process model structure*” [15]. Nevertheless, the investigation of this hypothesis for other architectures seems to be worthwhile [15], especially since LSTMs have issues with long sequences, as they pay elements that lie far behind less attention. The transformer architecture [21] bypasses this issue using an attention mechanism, assigning attention scores to each input part and prioritizing those with the highest scores as the most relevant, regardless of their position [1, 21]. The transformer architecture has spurred new state-of-the-art approaches across various research domains [21] (e.g., language models such as GPT-3 and BERT).

As this architecture also excels in PBPM to predict the control-flow in form of the next activity [1], this paper explores the following research question: *Do the attention scores of the transformer architecture give any insights whether the trained prediction model achieved an understanding of the control flow of a process?* Thus, our main contributions are: **(I)** We examine whether the aforementioned attention scores of the transformer architecture can serve as a solid basis for the development of XAI approaches. **(II)** We contribute two global, transformer-specific explanation approaches to investigate the explainability of the transformer’s attention mechanism and to offer types of explanation, which are motivated rather by the BPM domain than the AI domain. **(III)** Using eight data sets and five complementary metrics we evaluate the quality of the explanatory approaches, thereby offering a benchmark for sustainable comparability.

The rest of the paper is structured as follows: Sect. 2 provides background on PBPM (i.e., basic terminology, fundamentals of the transformer architecture). After delimiting our approach from related work (Sect. 3), a preliminary study investigates, whether attention scores can serve as an explanation (Sect. 4). Based on the result, we introduce two global explanation approaches (Sect. 5), systematically applied to various prediction models trained on real-life event logs (Sect. 6). Section 7 discusses potential limitations and concludes the paper.

2 Preliminaries

2.1 Basic Terminology

In the following A denotes the set of viable activities in a business process. The execution of a (business) process can be recorded in form of a *trace* $\sigma = \langle e_1, \dots, e_n \rangle$, i.e., a temporally ordered sequence of events associated with the same process instance. Each *event* represents the execution of an activity in the process and is characterized by event attributes, such as the name of the corresponding activity. Without loss of generality (since the considered process transformer only process activities), we denote traces in the following as sequences of their activity names. We denote with π_A a function that returns for a given event its executed activity and with $|\sigma|$ the length of the trace σ . The set of all traces pertaining to the same (business) process is referred as *(process) event log*. For PBPM approaches, prefixes of a trace are used to represent running process instances:

Definition 1 (Prefix). Let $\sigma = \langle e_1, \dots, e_n \rangle$ denote a trace and let be $r \in \{1, \dots, n-1\}$. The **prefix** of length r is a function hd which returns the first r elements of a trace: $hd(\sigma, r) = \langle e_1, \dots, e_r \rangle$.

This prefix, is then used as input to a function Ω predicting the next activity, given by $\Omega(hd(\sigma, r)) = \pi_A(e_{r+1})$. A PBPM approach, in this paper a transformer model, aims to learn the function Ω using a given event log.

2.2 Attention Mechanism

In deep learning the attention mechanism allows models to dynamically focus on different parts of a input sequence for generating output. The fundamental idea behind attention is to learn to assign different scores to the parts of the input, indicating how much focus the model should pay them when performing a task. Hence, it allows to prioritize most relevant information leading to more accurate and contextual output. The approach investigated in this paper is based on the so called *self-attention mechanism* [21] utilized within the transformer architecture for next activity prediction proposed by [1]. In the following, we describe parts of the architecture relevant for the paper at hand, i.e., the input, the self-attention mechanism, and the output of the transformer. For a description of the remaining parts of the architecture, we refer to [21] and [1], respectively.

According to [1] the transformer receives as input a list of activities (i.e., a prefix). This input is then embedded into a high-dimensional space of dimension d_k by converting each element of the list into a so called *embedding vector*. This representation leads to a continuous representation of the input and captures positional, semantic, and syntactic properties of the corresponding elements, and serves as input for the self-attention. It is important to note that the embeddings are automatically learned during training. In the attention mechanism an embedding vector x_i is used in three different ways: as *query*, *key*, and *value vector*. We get them by multiplying x_i with matrices W_Q , W_K , and W_V that

are learned during the training process, and pack them for more efficiency into matrices Q , V , and K (each column represents one vector) [21]. Next, we use Q , K , and V to calculate an attention score reflecting the importance of each element (query) under consideration in relation to the others. First, each element of the input sequence is scored against the currently considered element by taking the matrix product QK^T . The values of the resulting matrix are divided by $\sqrt{d_k}$ before applying a row-wise softmax operation to normalize the values to avoid slow training through vanishing gradients [21]. The obtained matrix after the softmax operation is called *attention score matrix* or *attention matrix* for short.

Finally each value vector is multiplied by the attention scores. The intent behind this step is, that if the attention value is very small, it eliminates the value vector due to irrelevance. Hence, the calculation can be summarized as:

$$Att(Q, K, V) = softmax \left(\frac{QK^T}{\sqrt{d_k}} \right) V.$$

To learn different types of relationships and patterns between the elements

of the input sequence, we use h independent and parallel attention mechanisms (called *attention heads* or *heads*) and combine them to so called *multi-head attention* [21]: $MultiHeadAttention(Q, K, V) = concat(h_1, \dots, h_h)W^O$ with $h_j = Att(QW_Q^j, KW_K^j, VW_V^j)$. In this formula the resulting matrix of each head j is arranged side by side and multiplied by a weight matrix W^O , that is learned during training. The process transformer presented in [1] uses four heads. The results of the multi-head attention are further processed and aggregated with different layers ending in a softmax output layer O [1]. This layer has a neuron for each activity in the process and accordingly returns a probability distribution p over the activities (also known as *prediction vector*). The activity with the highest probability is extracted as predicted next activity using the argmax function.

In the following let $\mathcal{M}$ be the trained transformer model. We denote with $p_\sigma = \mathcal{M}(\sigma)$ the models' outputted probability distribution for a prefix σ . Moreover, $p_\sigma(a)$ refers to the prediction score for activity $a \in \mathcal{A}$ in p_σ . With M_σ^j we denote the attention score matrix of head h_j received for prefix σ . To access a particular attention score within M_σ^j , we write $M_\sigma^j(i, k)$ to access the score in the i -th row and the k -th column. The vector $M_\sigma = (M_\sigma^1, \dots, M_\sigma^h)$ summarizes the attention score matrices of all heads in $\mathcal{M}$. For further investigations, it is necessary to understand the attention scores for the elements of the input sequence as a probability distribution. Therefore we flatten the attention score matrix M_σ^j for head j into a vector $v_\sigma^j = (M_\sigma^j(1, 1), M_\sigma^j(1, 2), \dots, M_\sigma^j(1, |\sigma|), M_\sigma^j(2, |\sigma|), \dots, M_\sigma^j(|\sigma|, |\sigma|))$ (concatenating the rows of M_σ^j). The flattened representation across all heads is then defined as $v_\sigma = (v_\sigma^1, \dots, v_\sigma^h)$. Based on the flattened representations we define

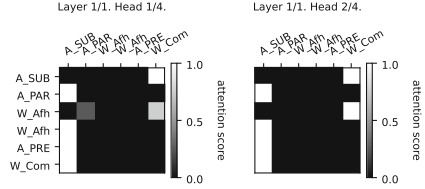


Fig. 1. Attention scores of two different heads for a prefix of the BPIC12 event log.

the *head attention score distribution* for head j obtained from σ as $\alpha_{\sigma}^j = \frac{v_{\sigma}^j}{\|v_{\sigma}^j\|}$, and the *attention score distribution* across all heads as $\alpha_{\sigma} = \frac{v_{\sigma}}{\|v_{\sigma}\|}$, where $\|\cdot\|$ denotes the L1-norm, i.e., the component-wise sum of the elements.

2.3 Interpretation of Attention Scores

Figure 1 depicts attention score matrices for two heads, illustrating how elements of an input sequence (here the activities of the events) relate to each other through the model’s attention mechanism. The interpretation of an attention score matrix is identically for all heads. In the attention score matrix, rows and columns correspond to elements of the input sequence, with rows representing these elements as queries and columns as keys. By examining a row, we can assess the importance the transformer model assigns to each other element in the input sequence from the perspective of the row’s element. In contrast, analyzing a column reveals the degree of attention all other elements in the input sequence pay to the element associated with that column. Within the matrix higher attention scores appears lighter, while lower scores are shown darker, indicating the strength of the relationship between the elements in the respective row and column.

3 Related Work

In the following we briefly discuss XAI approaches in the PBPM domain with focus on next activity prediction. They can roughly be classified into *model-specific* or *model-agnostic* approaches. While model-specific approaches are tailored to the inner working of a model, model-agnostic approaches can be applied to any machine learning model, regardless of its internals. Although model-agnostic approaches are highly versatile and universally applicable, they provide less detailed insights in the decision processes of the particular model. The survey [20] identifies 20 XAI approaches in the PBPM domain and categorizes them along different dimensions. It shows, that the vast majority of approaches is model-agnostic and limited to local explanations. Hence, there is a lack of global model-specific explainer. When combining criteria of model-specific and next-activity explainers, only three papers remain [8, 18, 22], that can be considered as directly related work. None of these used the transformer as their specific model, however [18] uses an attention mechanism within an LSTM, whereas this mechanism differs from those used by the transformer from [1]. The results in this approach show accumulated attention scores for each position in a prefix. Although the authors refer to the work of [17] which states that attention cannot be interpreted “as is”, they do not further consider tests proposed in [17] to verify and critically reflect whether the observed attention scores are truly interpretable. Hence, it is still an open question in the PBPM domain, whether attention scores can be trusted as basis for explanation [10, 23]. We address this question in Sect. 4. The *Explainable Next Activity Prediction (XNAP)* approach proposed in [22] provides explanations for LSTMs for next activity prediction using layer-wise relevance propagation resulting in assigning each activity in the

input prefix a relevance score. However, just as in [18] they do not conduct any quantitative evaluation and limit themselves to a qualitative analysis of the results. The approach proposed in [8] also relies on LSTM networks but differs from [18] and [22] by outputting a directed and weighted graph as explanation instead of relevance score for elements in the input prefix. In the resulting graph, activities are represented by nodes and the probabilities on each edge (u, v) indicate the likelihood of the transition from u to v as predicted by the model. Hence, this graph can be considered only as a Directly Follows Graph enriched with probabilities. While this approach reflects which edges can be traversed according to the LSTM model, it is questionable whether the graph can explain the LSTM’s decision-making. Beside this direct related work there are some further noteworthy approaches to be shortly discussed as it serves as inspiration for the developed approaches within this paper. The LORE [7] approach approximates a black box model locally by generating local instances around the input instance. In this generation process the instances are mutated, i.e., changing continuous or categorical values. The LORELEY [9] approach extends the LORE approach [7] and applies it to next-activity prediction in the PBPM domain by restricting mutation options to avoid violating the control flow. However, it is a model-agnostic approach and therefore not comparable to the approach presented in this paper.

Moreover, we distinguish our work from distantly related approaches that propose local methods or focus on other prediction targets. For example, Rizzi et al. [16] identify features responsible for wrong outcome-predictions using the model-agnostic local explainers LIME and SHAP. By reducing the impact of the identified features, the accuracy of the prediction model can be increased. More general guidelines for the use of features are determined in the work of Stevens et al. [19] that reveals for outcome-prediction that a small number of features that are as much as possible independent are preferable without compromising on faithfulness. In addition, Elkawaga et al. [5] emphasise the importance of investigating the extent to which explanations reflect the characteristics of the data. Their findings indicate that feature selection is crucial for both improving prediction quality and obtaining reliable explanations. However, since the transformer investigated in our work, exclusively utilises the activity event attribute, feature selection and feature importance do not apply for creating explanations. This is one of the main reasons why a tailored XAI approach for this architecture is required. While the aforementioned approaches primarily rely on input perturbations, [12] proposes a local explanation approach for outcome prediction that defines local regions in the latent space of the deep learning model via clustering and decision trees.

4 Pre-Study: The Relevance of Attention Scores

Before using attention scores as pivotal component in an explanation approach for next activity prediction, we verify their reliability in this context, i.e., that they indeed highlight activities decisive for a prediction. We investigate whether

Table 1. Statistics of the event logs ($|\sigma|$ is the trace length). We focused only on control-flow relevant characteristics, since the transformer model only considers activities.

Event Log	#Cases	#Act.	#Events	AVG. $ \sigma $	Max. $ \sigma $	#Variants
BPIC12	13087	24	262200	20.04	175	4366
BPIC12_O	5015	7	31244	6.23	30	168
BPIC12_W	9658	7	170107	17.61	156	2643
BPIC12_WC	9658	6	72413	7.50	74	2263
BPIC13_CP	1487	4	6660	4.48	35	183
BPIC13-I	7554	4	65533	8.68	123	1511
Helpdesk	4580	14	21348	4.66	15	226
Sepsis	1050	16	15214	14.49	185	846

attention scores in general affect the prediction (Exp. 1) and shed light on the extent to which particular attention scores contribute to a prediction (Exp. 2).

Datasets: For our study we use eight frequently used real-world event logs (see Table 1) obtained from the *4TU Center for Research Data*¹. The BPIC12 datasets (O, W, and WC) are obtained from the BPIC12 event log by different filtering operations. While BPIC12_O is derived from BPIC12 by only keeping events whose activities start with “O_”, BPIC12_W only contains events whose activities start with prefix “W_”. The BPIC12_WC is a refinement of BPIC12_W only keeping events whose lifecycle attribute is set to “complete”.

General Experiment Setup: All sub-experiments in this study share some similarities in the setup. We divide each event log randomly into training and test data in a ratio of 70% training to 30% testing data. To ensure reproducibility of our experiments, seeds are set. To create specific test samples, we extract all prefixes with a minimum length of 1 from the traces in the test data. The transformer model is trained on the training data using the standard hyperparameters as defined in [1]. The test examples are then used in the experiments to obtain predictions, in which the influence of the attention scores is examined. For comparing attention score distributions and prediction vectors, we use two measurements. For quantifying the divergence between two attention distributions α_1 and α_2 , we use the *Jensen-Shannon-Divergence* (JSD) [10] that is based on the *Kullback Leibler Divergence* (KL) defined in [11]:

$$JSD(\alpha_1, \alpha_2) = \frac{1}{2} KL(\alpha_1 \parallel \frac{\alpha_1 + \alpha_2}{2}) + \frac{1}{2} KL(\alpha_2 \parallel \frac{\alpha_1 + \alpha_2}{2}).$$

The change between two prediction vectors p_1 and p_2 is measured as the *Total Variation Distance* (TVD):

$$TVD(p_1, p_2) = \frac{1}{2} \cdot \sum_{i=1}^{|A|} |p_1^i - p_2^i|, \text{ where } p^i \text{ denotes the } i\text{th component of } p.$$

¹ <https://data.4tu.nl/>.

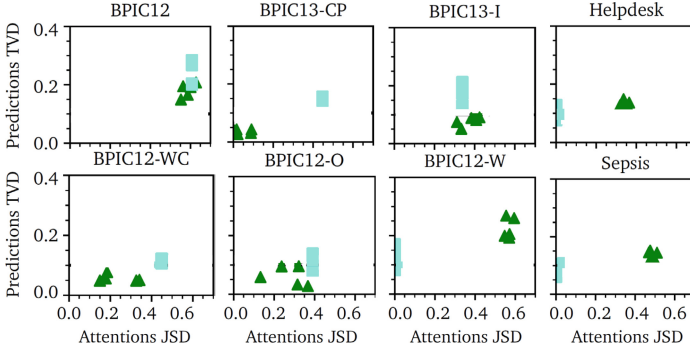


Fig. 2. JSD vs. TVD plots. Rectangles = models with frozen weights, triangles = seeded base models. JSD can only take values between 0 and $\log(2) = 0.693$.

Experiment 1 – Attention Mechanism Parameter Manipulation: We first investigate whether attention is necessary in the first place, following the experimental setup in [23]. We thus train a baseline model $\mathcal{M}_b$ with randomly initialized attention parameters (i.e., matrices W_Q , W_K , W_V , and W_O) and a modified one $\mathcal{M}_m$ identical to the training setup of $\mathcal{M}_b$ except that we freeze the attention parameters to uniform weights during training. We repeat this procedure five times with differently initialized baselines and modified models to minimize the influence of chance. To ensure to get indeed different initializations, we use random seeds. Then each manipulated transformer $\mathcal{M}_m$ is compared against the corresponding baseline model $\mathcal{M}_b$. To do this, we send all test samples through all models and obtain for each sample and model the corresponding attention distribution as well as the prediction vector. Then for each sample we compare attention distribution and prediction vector obtained from $\mathcal{M}_b$ with those obtained from each modified model using JSD and TVD, respectively. The computed JSD and TVD values are afterwards averaged across all samples. The intention behind this setup is, that the higher the divergence in attention scores (larger JSD) and the smaller the divergence in output (lower TVD), the less they affect the prediction [10, 23]. Therefore, if the same prediction can be made with completely different attention scores, they can hardly serve as explanation.

For interpretation, we plot the mean JSD against the mean TVD achieved for each model (see Fig. 2). Models positioned rightward and lower in the plots indicate less reliable attention scores. Hence, most of the event logs (BPIC12-W, Helpdesk, Sepsis) show good results. In the case of a right tendency (BPIC12), the TVD values are clearly greater than zero. Consequently, the attention scores can be trusted using the transformers trained on these event logs.

Experiment 2 – Attention Score Masking: In a second experiment, we investigate how particular attention values affect the prediction (see Fig. 3). Therefore, we once mask elements in the input prefix and once the corresponding attention scores in the heads’ attention score matrices and compare the model outputs via TVD. We mask elements in the prefix by replacing them by a padding

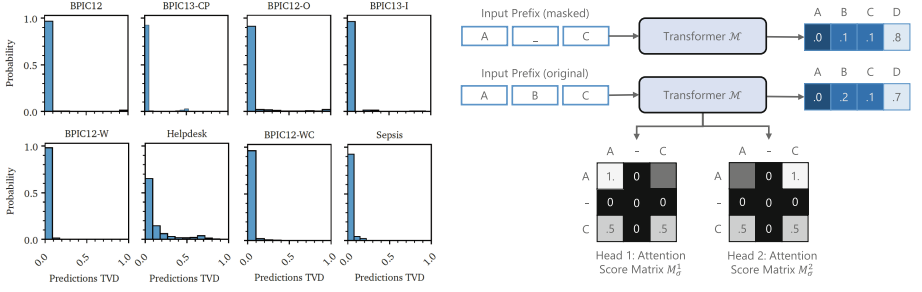


Fig. 3. *Left:* Variation between predictions TVD between masked elements in the prefix and only masked attention scores matrix. *Right:* Used masking variants.

symbol ($_$) that indicates the model that there is no element. In contrast, masking the attention scores is done by setting all values of the corresponding rows and columns in the heads' attention score matrices to zero. Again we use all test samples and mask in each sample one element after the other. For each sample we obtain a prediction vector p_m for the masked sample and a prediction vector p_{am} for the unmasked sample that was processed with masked attention matrix. We compare p_m and p_{am} computing TVD. Low TVD values indicate that it does not matter whether we mask an element in the trace or in the attention score matrices. Hence, changes in the attention score matrices approximately affect the output in the same way as changes in the input trace. From this it can be concluded that particular attention values have relevance and can serve as an explanation. We observe that the histograms of TVD values (see Fig. 3) are heavily left-leaning, i.e., masking directly in the prefix leads to almost the same prediction as masking in the attention score matrices. Minor deviations from this behavior are only noticeable in the histograms of the Helpdesk log.

5 Explanation Approaches

In this section we propose two global explainers that receive as input the trained transformer prediction model $\mathcal{M}$ and a set of prefixes L to be explained. Both explainer construct a comprehensible directed graph describing the control flow of the process that acts as an indicator to which extent the PBPM model understands the control-flow of the process. Since attention scores reliably predict the next activity (cf. Section 4), we can base our explanation approach on them.

5.1 Backward Explainer

The first explainer derives explanations for individual traces, subsequently integrating them into an overarching explanation for L via a directed graph G . This process is explained in the remainder of this section and involves creating a local graph G_σ for each prefix $\sigma \in L$, which is then merged directly into G .

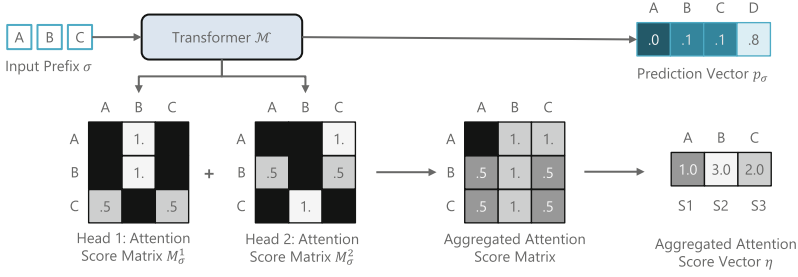


Fig. 4. Determining aggregated attention scores for each event in the prefix, assuming that $\mathcal{M}$ possesses two heads.

Creating Local Graph: A crucial step for creating a local graph G_σ for a prefix σ is to use the head attention scores to identify the relevant activities $\mathcal{A}_r$ for the prediction p_σ . To do this we proceed as follows: We consider an activity as relevant for prediction p_σ , if its aggregated attention score over multiple modifications of σ is large enough. To obtain different modifications of σ , we randomly apply modification operations. We pass each modification σ_m to $\mathcal{M}$ to obtain prediction vectors p_{σ_m} and the heads' attention score matrices $M_{\sigma_m}^i$. Then the cosine distance between p_{σ_m} and p_σ is computed and compared with an a-priori defined threshold δ_{sim} , to check whether the prediction for σ_m is still close to the prediction of σ . If σ_m satisfies this condition, the attention scores for each element j in σ_m are aggregated across the different heads:

$$S_j = \sum_{n=1}^{|\sigma_m|} \left(\sum_{i=1}^n M_{\sigma_m}^i \right)_{nj}.$$

In this formula first the heads' attention score matrices $M_{\sigma_m}^i$ are component-wise summed, and afterwards the j th column is summed. Hence, we get for each σ_m an aggregated attention score vector $\eta_{\sigma_m} = (S_1, \dots, S_{|\sigma_m|})$ containing a total attention score for each event. This procedure is illustrated in Fig. 4. Next this total values are further aggregated per activity to a vector ψ by summing up the total attention scores of each event belonging to a certain activity. Then ψ_σ is normalized so that its components are within range $[0, 1]$. We denote with $\psi_\sigma(a)$ the total attention score for activity a . Finally, we filter the activities by a given threshold δ_{attr} by keeping only activities with $\psi_\sigma(a) > \delta_{attr}$ to get only those activities that have a particularly high total attention score. This filtering is needed as irrelevant elements most have a small but not zero attention score and thus would be considered otherwise. The identified relevant activities $\mathcal{A}_r$ and the most likely next activities $\mathcal{P}_r$ obtained from prediction vector p_σ (i.e., activities a with $p_\sigma(a) > \delta_{pred}$). We do not use a fixed number of activities from p_σ , since depending on the process, in some cases the model is nearly sure that only one activity is possible, while in other cases (e.g., by an AND split) several activities seem valid to be executed next. Afterwards, edges between

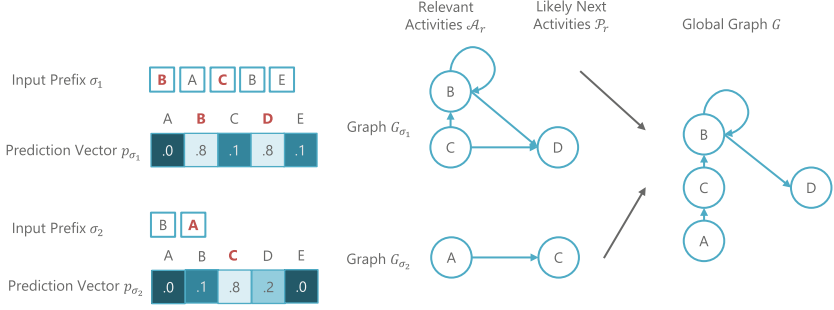


Fig. 5. Example for BackwardExplainer. Relevant activities and likely next activities are highlighted bold faced in red color in the input prefix and prediction vector. (Color figure online)

all nodes from $\mathcal{A}_r$ and $\mathcal{P}_r$ are established. Let us illustrate the procedure on the following example (see Fig. 5). We have given prefix $\sigma_1 = \langle B, A, C, B, E \rangle$ where the aggregated attention scores identified activities B and C as relevant for the prediction. Moreover, we obtain a prediction vector p_{σ_1} indicating that either activity B or C occur likely next. Hence, we get a local graph $G_{\sigma_1} = (\{B, C, D\}, \{(C, B), (B, B), (B, D), (C, D)\})$. The graph can then be read as follows: For executing activity D it is decisive that activities B and C were executed before.

Joining Local Graphs: Each local graph G_σ is directly integrated into the global graph G . Therefore, vertices and edges of G_σ are inserted into G if not yet included. Assume that in our example in Fig. 5, we have now a second local graph $G_{\sigma_2} = (\{A, C\}, \{(A, C)\})$. Then the final graph would be $G = (\{A, B, C, D\}, \{(C, B), (B, B), (B, D), (C, D), (A, C)\})$. After melting G_σ into G , we perform a pruning step to remove undesired shortcuts that are likely not to occur in the underlying process. Such undesired shortcuts emerge, if an activity in the front part of σ receive large attention for the prediction but activity and predicted activity do not follow each other directly. Hence, we remove an edge (u, v) if edges (u, a_n) and (a_n, v) exist, with a_n being the last activity in σ .

5.2 Attention Exploration Explainer

The Backward Explainer has one major drawback: It directly determines the edges of the graph when analysing a prefix. Once inserted an edge is retained, except if it is a shortcut that can be removed in the pruning step. Thus, existing edges are not updated based on information obtained from further prefixes. We address this issue, by no longer relying on local graphs. Instead, we calculate a new relevance score for each activity across all prefixes presented to the algorithm. The decision about inserting an edge is postponed to the end of the procedure after inspecting all prefixes using the final relevance scores of the activities.

Determining Relevance: To prevent relevance scores from increasing with each additional prefix and to reduce them when the attention score of an activity is consistently low, we introduce negative attention scores. In the transformer architecture, attention scores are inherently non-negative, with lower values indicating minimal impact. We encode the latter with a negative value. A positive value supports edge insertion, while a negative value is against it.

Therefore we first determine for each prefix $\sigma \in L$ a so called *score matrix* K_σ , with a column and a row for each activity $a \in A$. First, we extract for σ the relevant activities $\mathcal{A}_r$ in the same way as in the Backward Explainer above. Additionally, we examine the positions of the relevant activities in σ and store them in a set $\mathcal{I}_\sigma$. Next, we use all subsets of $\mathcal{I}_\sigma$ within two masking scenarios to quantify the individual and collective impact of activities on the prediction. For each subset $r \in \mathcal{I}_\sigma$, we consider two scenarios. In the “masking out most” scenario we mask the unimportant activities, i.e., all positions that are not contained in r . In contrast the “masking out a few” scenario masks the important activities, i.e., all positions in r . To obtain values for our score matrix K_σ , we now compare σ with all of its masked versions (denoted as σ_m). Therefore, we send σ and σ_m through $\mathcal{M}$ to obtain prediction vectors p_σ and p_{σ_m} respectively. As well as the most likely next activities $\mathcal{P}_r$ for prefix σ . Then, we compute as in the Backward Explainer the per activity aggregated attention score vectors ψ_σ and ψ_{σ_m} indicating how much focus the model gives to an activity in the normal and masked trace, respectively. The computation of the scores is described in detail in Algorithm 1. In this calculation, a score is calculated for each activity in the masked prefix regardless of whether it is masked or not. For masked activities, the score is the product of the prediction score and attention scores. If the prediction for a masked activity matches its prediction in the normal trace, the score is negated, indicating reduced relevance due to the activity’s negligible impact on the prediction. For non-masked activities, scores are derived differently: if an activity’s prediction value is unchanged with masking, the score is the product of its attention score and normal trace prediction. If the prediction changes, the score is based on the product of the absolute differences in both attention scores and predictions, reflecting the influence of the activity in the masked trace. Eventually the calculated score is added to the corresponding cell (row for the predicted activity and column the masked activity) in the score matrix.

From Algorithm 1 we receive a score matrix for each masked version of σ , which are summed to obtain two scenario-specific score matrices, K_σ^{few} and K_σ^{most} . The two matrices comprehensively assess the relevance of each activity in σ , considering the impact of its presence or absence in different combinations.

Building the Explanation Graph: Finally we must aggregate the score matrices of both scenarios we have obtained for each prefix:

$$K^{few} = \sum_{\sigma \in L} K_\sigma^{few} \quad K^{most} = \sum_{\sigma \in L} K_\sigma^{most}.$$

Algorithm 1: computeRelevanceScore

Data: Prefix σ , Masked prefix σ_m , aggregated attention scores per activity ψ_σ and ψ_{σ_m} , prediction vectors p_σ and p_{σ_m} , most likely next activities $\mathcal{P}_r$

Result: Score matrix

$K \leftarrow (0)_{ij} \forall i, j \in \{1, \dots, |A|\}$

foreach $a \in \mathcal{P}_r$ **do**

foreach *masked activity* $a_m \in \sigma_m$ **do**

$s \leftarrow p_\sigma(a) \cdot \psi_\sigma(a_m)$

if $p_\sigma(a) \sim p_{\sigma_m}(a)$ **then**

$s \leftarrow -s$

end

$K(a, a_m) \leftarrow K_\sigma(a, a_m) + s$

end

foreach *non-masked activity* $a_n \in \sigma_m$ **do**

if $p_\sigma(a) \sim p_{\sigma_m}(a)$ **then**

$s \leftarrow \psi_{\sigma_m}(a_n) \cdot p_\sigma(a)$

else

$s \leftarrow |\psi_\sigma(a_n) - \psi_{\sigma_m}(a_n)| \cdot |p_\sigma(a) - p_{\sigma_m}(a)|$

end

$K(a, a_m) \leftarrow K_\sigma(a, a_m) + s$

end

end

return K

Afterwards we normalize each row in K^{few} and K^{most} (i.e., the entry in the row should sum up to one) to ensure comparability across different activities. Then, each entry of both matrices is converted to a boolean value, by checking whether it exceeds a predefined threshold δ_{edge} (true) or not (false). While “true” indicates a significant relevance of an activity on another, “false” indicates that there is no relationship between the activities. Eventually, we combine the insights from both masking scenarios by a component-wise logical OR operation between K^{few} and K^{most} . The resulting matrix K can be interpreted as adjacency matrix, where a “True” value indicates an directed edge from the corresponding activity of the column to the corresponding activity of the row. A “False” value indicates no edge. Finally, we construct the graph from this adjacency matrix.

6 Evaluation

In this section, we examine the performance of our explainers using the event logs from the pre-study. Our approach is implemented as prototypical Python framework that contains both explainers and all methods required for reproducibility². Since there are no other transformer-specific global explainers available, we cannot compare fairly with other approaches. A comparison with existing local or model-agnostic approaches would not be meaningful, as these pursue completely

² Source code: <https://github.com/mkaep/transformer-explainability>.

Table 2. Results for Attention Exploration Explainer (AE) and Backward Explainer (BE); cells: mean (first), std. deviation (second); N = no value could be computed, bold faced = best values; compactness: length of right-hand side.

	Correctness		Completeness		Continuity		Contrastivity		Compactness	
	BW	AE	BW	AE	BW	AE	BW	AE	BW	AE
BPIC12	0.63 ±0.20	0.65 ±0.15	0.06 ±0.01	0.07 ±0.01	0.71 ±0.05	0.93 ±0.05	0.45 ±0.33	0.26 ±0.25	1.66 ±0.20	4.12 ±0.77
BPIC12_O	0.48 ±0.22	0.48 ±0.22	0.13 ±0.02	0.21 ±0.02	0.95 ±0.07	0.95 ±0.07	0.07 ±0.18	0.07 ±0.18	2.56 ±0.06	2.21 ±0.09
BPIC12_W	0.65 ±0.21	0.65 ±0.21	0.10 ±0.08	0.27 ±0.04	0.81 ±0.13	0.92 ±0.06	0.19 ±0.30	0.16 ±0.26	0.90 ±0.14	1.86 ±0.25
BPIC12_WC	0.62 ±0.18	0.56 ±0.18	0.29 ±0.10	0.28 ±0.09	0.90 ±0.06	0.89 ±0.05	0.03 ±0.13	0.02 ±0.09	1.48 ±0.60	1.89 ±0.15
BPIC13_CP	0.75 ±0.18	0.60 ±0.28	0.50 ±0.14	0.79 ±0.15	0.49 ±0.13	0.98 ±0.10	0.05 ±0.20	0.05 ±0.19	0.19 ±0.24	2.09 ±0.35
BPIC13-I	N ± N	0.57 ±0.25	0.29 ±0.04	0.77 ±0.08	0.19 ±0.24	0.99 ±0.02	0.04 ±0.18	0.03 ±0.12	0.19 ±0.24	2.00 ±0.09
Sepsis	0.86 ±0.13	0.59 ±0.17	0.04 ±0.02	0.15 ±0.02	0.57 ±0.09	0.92 ±0.13	0.17 ±0.33	0.13 ±0.27	0.83 ±0.16	2.76 ±0.42
Helpdesk	0.91 ±0.08	0.73 ±0.18	0.36 ±0.05	0.30 ±0.04	0.73 ±0.15	0.85 ±0.19	0.08 ±0.22	0.20 ±0.30	1.99 ±0.39	2.55 ±0.62
AVG	0.7	0.60	0.22	0.36	0.67	0.93	0.14	0.12	1.23	2.44
AVG(AE-BW)	-0.01		0.134		0.26		-0.02		1.21	

different objectives and global explainability is a much more difficult task. We first evaluate the quality of the models to rule out the possibility that they have not learned any skills at all. Only then, we evaluate the explainers, relying on quantitative metrics to avoid any subjective biases [13]. This principle reduces the risk that our explainers will be blamed for the errors of the black box models.

We trained and evaluated the models following the setup in the pre-study. For evaluation we score a model by a commonly used weighted F1-Score [1, 2, 14]. We achieved a mean weighted F1-score across all event logs of 0.74 (minimal value 0.46 and maximal value of 0.93). Hence, there are no no-skill models and it is likely that attention scores have been properly trained. Significantly higher values are not achievable as the transformer is limited to the activity event attribute. Consequently, many cases are undecidable and are therefore predicted purely statistically on the basis of frequency of occurrence. However, incorrect predictions are unproblematic for our explanation approaches, as they are intended to elucidate the rationale behind the model’s decision. This is particularly, but not exclusively, useful for a more precise error analysis (Sect. 7).

For our quantitative evaluation, we utilize various metrics from [13]. These metrics rely on explanations in form of rule sets, because of which it is necessary to transform our graphs into rule sets. Therefore, each vertex v leads to a rule reflecting its direct successors S_v . Hence, we obtain a set of rules of the form $R : v \rightarrow S_v$ for each v in the graph’s vertices V . Note, that all rules have exactly one activity on the left-side. The rule can be interpreted that right sides include all possible predictions, given the left-hand side. In the following, we give a brief overview of the intent of the used metrics and refer for more details to [13]:

Correctness: This metric evaluates whether the given explanation is truthful [13]. Therefore one input feature (in our case an activity) at a time is masked to determine its importance for model and explainer. Finally, the correlation between the feature importance for the model and for the explainer is calculated. Hence, the metric takes values within $[-1, 1]$, where large values indicate that the explainer primarily uses the correct feature for its explanations.

Completeness: This metric indicates how well explanations match the predictions of the transformer [13]. Transformer predictions are considered ground truth and the right-hand sides of the explanation rules are evaluated as predictions. From the resulting (multi-label) confusion matrix we derive the F1-Score.

Continuity: This metric evaluates if the XAI approach provides similar explanations for slightly modified inputs and how well the approach generalizes w.r.t. unseen inputs [13]. Varying the input it is checked how much the new explanation deviates [13]. Values range between 0 and 1 (perfect continuous prediction) [13].

Contrastivity: Contrastivity represents the counterpart to continuity and evaluates how different the explanations of an XAI approach are for dissimilar inputs [13]. Here, a value of 1 signals perfect contrastivity.

Compactness: This metric evaluates the brevity of an explanation by counting the rules and averaging the lengths of their right-hand sides. Per [13], a more compact (shorter) explanation is preferable due to human cognitive limitations.

To apply the metrics, a consistent percentage of prefixes from each event log is randomly chosen to accommodate their varying sizes. Metrics are computed for each prefix individually, followed by averaging these values. Table 2 presents these average values together with their standard deviations. We observe consistently high correctness values, which clearly indicate that high attention scores actually correlate with the activities that are decisive for the prediction. Only the BPIC12_O log falls slightly. A more heterogeneous picture emerges in case of the completeness metric where, depending on the event log, relatively poor values (BPIC12) or quite good values (BPIC13) are achieved. A deeper investigation revealed that the low values are caused by low recall values. Given the significantly higher precision values, the given explanations are often correct, however not all explanations are found. In the outputted graphs (or rules) it means that the edges are correct (high precision), but some edges are missing (low recall). Finally, the consistently very high continuity values suggest that the explainers are very robust, i.e., they also provide meaningful explanations for previously unseen data. However, the simultaneously very low contrastivity values suggest that the changes in the input have hardly any effect on the prediction. This is essentially caused by the fact, that the transformer only learns the most frequent execution paths due to its limitation to the event activity attribute, i.e., it reacts to strong changes in a prefix by providing a “standard” prediction. For a holistic comparison that indicates the performance of our explainers across the different datasets, we aggregated the results as follows. First, we calculate for each event log the difference between the value of the particular metric achieved by the Attention Exploration Explainer and the Backward Explainer. Afterwards

we average all difference values to get a measure of the performance differences across the event logs (cf. last row of Table 2). We see that across all considered event logs the Backward Explainer and the Attention Exploration Explainer perform nearly identical in terms of correctness and contrastivity. Nevertheless, there are some differences for particular event logs, e.g., in case of contrastivity for BPIC12 and Helpdesk) as well as in correctness for BPIC13_CP, Sepsis and Helpdesk. This slightly poorer performance on these event logs for correctness metric is attributed to its higher recall values in the completeness metric. This results in the Attention Exploration Explainer identifying more explanations that are occasionally erroneous. This is particularly problematic in variant-rich processes (Sepsis), where explanations are often applicable to only a few traces, and event logs with concept drifts (Helpdesk), where explanations may lose their validity over time. For completeness and continuity, however, the Attention Explainer performs noticeably better. Notwithstanding the higher compactness values for the Attention Exploration Explainer, they remain within a low range (<5) and therefore do not impede interpretability. In contrast, the Backward Explainer’s low compactness values are one of the reasons for performing worse in completeness and continuity. As superior compactness of explanations is of little value in the absence of strong correctness, completeness, and continuity, it can be considered as a secondary metric. In conclusion, the Attention Exploration Explainer can be considered the superior option in the overall evaluation.

7 Concluding Remarks

This paper presents two transformer specific XAI approaches based on attention scores for next-activity prediction. We show first, that attention scores indeed form a reliable basis for explanations and presented two approaches explaining predictions through graphs. These graphs show how attention scores for activities relate to predictions. The evaluation using various quantitative metrics demonstrates that the Attention Exploration Explainer outperforms the simpler Backward Explainer. This provides compelling evidence that explainers based on attention scores hold significant potential. This offers a wide range of useful opportunities for the future development and application of transformer-based predictions of the next activity. First, it provides transparency in the decision-making process of the model increasing the trust of process participants in the predictions. Second, the global explanatory capabilities of the presented approaches allow to assess the process understanding in general and to avoid contradictory explanations across different samples as in many local explainers. Third, the insights provided by the explainers, particularly in instances of erroneous predictions, offer a valuable starting point for model improvement, which may be achieved through enhanced training or the incorporation of additional training data. Moreover, an analysis of the graph structure can provide indications of potential weaknesses that may lead to the generation of erroneous predictions.

Therefore, we answer our research question (Sect. 1) as follows: the transformer’s attention scores are effective for checking and visualizing its process

understanding. However, since the trained prediction models only incorporate the activity event attribute, the process cannot be fully learned inherently.

Limitations: Real-life event logs often suffer from quality issues such as noise, incompleteness, and inconsistencies, affecting the prediction and consequently the explanation quality in a negative way. In our pre-study we aggregate the attention scores of all heads to take a holistic perspective on the attention scores. Thus, it is unexplored what particular heads learn and how they potentially differ. In future work this should be investigated more in detail. Currently, the proposed explainers employ various thresholds to filter out less relevant relationships affecting the edges in the explanation graph. Thus, optimizing these thresholds holds vast potential to increase the explanation capabilities. An interesting starting point would be to adjust thresholds flexibly depending on the prefix considered.

Future Work: In addition to the points already mentioned for future work, the used masking and modification operations within the relevance determination should be improved by using more advanced operations like in the LORELEY approach. Also the generation of process models out of prediction models should be consequently fostered as a novel way of process discovery.

References

1. Bukhsh, Z.A., Saeed, A., Dijkman, R.M.: Processtransformer: predictive business process monitoring with transformer network (2021)
2. Camargo, M., Dumas, M., González-Rojas, O.: Learning accurate LSTM models of business processes. In: Hildebrandt, T., van Dongen, B.F., Röglinger, M., Mendling, J. (eds.) BPM 2019. LNCS, vol. 11675, pp. 286–302. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26619-6_19
3. Carvalho, D.V., Pereira, E.M., Cardoso, J.S.: Machine learning interpretability: a survey on methods and metrics. *Electronics* **8**(8) (2019)
4. Di Francescomarino, C., Ghidini, C.: Predictive Process Monitoring, pp. 320–346. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-08848-3_10
5. Elkhawaga, G., Abu-Elkheir, M., Reichert, M.: Explainability of predictive process monitoring results: can you see my data issues? *Appl. Sci.* **12**(16) (2022)
6. Evermann, J., Rehse, J.R., Fettke, P.: Predicting process behaviour using deep learning. *Decis. Support Syst.* **100**, 129–140 (2017)
7. Guidotti, R., Monreale, A., Ruggieri, S., Pedreschi, D., Turini, F., Giannotti, F.: Local rule-based explanations of black box decision systems (2018)
8. Hanga, K.M., Kovalchuk, Y., Gaber, M.M.: A graph-based approach to interpreting recurrent neural networks in process mining. *IEEE Access* **8**, 172923–172938 (2020)
9. Huang, T.H., Metzger, A., Pohl, K.: Counterfactual explanations for predictive business process monitoring. In: Themistocleous, M., Papadaki, M. (eds.) Information Systems. EMCIS 2021. LNBIP, vol. 437, pp. 399–413. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-95947-0_28
10. Jain, S., Wallace, B.C.: Attention is not explanation. In: Proceedings of the 2019 Conference of the North American Chapter of the ACL. ACL, Minneapolis, June 2019

11. Kullback, S.: An application of information theory to multivariate analysis. *Ann. Math. Stat.* **23**(1), 88–102 (1952)
12. Mehdiyev, N., Fettke, P.: Explainable artificial intelligence for process mining: a general overview and application of a novel local explanation approach for predictive process monitoring. In: Pedrycz, W., Chen, S.-M. (eds.) *Interpretable Artificial Intelligence: A Perspective of Granular Computing*. SCI, vol. 937, pp. 1–28. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-64949-4_1
13. Nauta, M., et al.: From anecdotal evidence to quantitative evaluation methods: a systematic review on evaluating explainable AI. *ACM* **55**(13s) (2023)
14. Pasquadibisceglie, V., Appice, A., Castellano, G., Malerba, D.: Using convolutional neural networks for predictive process analytics. In: *International Conference on PM* (2019)
15. Peeperkorn, J., Broucke, S.v., De Weerd, J.: Can recurrent neural networks learn process model structure? *J. Intell. Inf. Syst.* **61**(1), 27–51 (2023)
16. Rizzi, W., Di Francescomarino, C., Maggi, F.M.: Explainability in predictive process monitoring: when understanding helps improving. In: Fahland, D., Ghidini, C., Becker, J., Dumas, M. (eds.) *BPM 2020. LNBIP*, vol. 392, pp. 141–158. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58638-6_9
17. Serrano, S., Smith, N.A.: Is attention interpretable? In: *Proceedings of the 57th Annual Meeting of the ACL*, pp. 2931–2951. ACL, Florence, Italy, July 2019
18. Sindhgatta, R., Moreira, C., Ouyang, C., Barros, A.: Exploring interpretable predictive models for business processes. In: Fahland, D., Ghidini, C., Becker, J., Dumas, M. (eds.) *BPM 2020. LNCS*, vol. 12168, pp. 257–272. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58666-9_15
19. Stevens, A., De Smedt, J., Peeperkorn, J.: Quantifying explainability in outcome-oriented predictive process monitoring. In: Munoz-Gama, J., Lu, X. (eds.) *ICPM 2021. LNBIP*, vol. 433, pp. 194–206. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-98581-3_15
20. Stierle, M., Brunk, J., Weinzierl, S., Zilker, S., Matzner, M., Becker, J.: Bringing light into the darkness - a systematic literature review on explainable predictive business process monitoring techniques. In: *ECIS* (2021)
21. Vaswani, A., et al.: Attention is all you need (2017)
22. Weinzierl, S., Zilker, S., Brunk, J., Revoredo, K., Matzner, M., Becker, J.: XNAP: making LSTM-based next activity predictions explainable by using LRP (2020)
23. Wiegrefe, S., Pinter, Y.: Attention is not not explanation. In: *Proceedings of EMNLP-IJCNLP. ACL*, November 2019